



**l'école d'ingénierie
informatique**

CERTIFICATION DÉVELOPPEUR EN INTELLIGENCE ARTIFICIELLE ET DATA SCIENCE RNCP 36581

BLOC DE COMPÉTENCES E6.2 : Développer un modèle prédictif d'une solution I.A

Développement et déploiement d'une application dans le respect du cahier des charges Client // Développement applicatif utilisant une API IA

SOMMAIRE

Introduction

Contexte et Objectifs

- I. Contexte
- II. Fonctions principales
- III. Contraintes et exigences

Rappel sur les données utilisées

Modèle IA développé

- I. Algorithmes choisis
- II. Préparation des données
- III. Entraînement et sauvegarde
- IV. Métriques de performance

Intégration dans l'API FastAPI

- I. Architecture de l'API
- II. Endpoint de prédiction
- III. Documentation interactive

Application Front-End

- I. Choix techniques
- II. Fonctionnalités de l'interface
- III. Accessibilité et ergonomie
- IV. Tests automatisés

Benchmark des solutions

Gestion de projet et méthodologie

Conclusion

Annexes

Introduction

Dans le cadre de la MSPR 01, nous avons mis en place une chaîne **ETL** pour collecter, nettoyer et visualiser des données historiques de la pandémie du VIH, via des scripts Python et un dashboard Power BI.

Pour le MSPR 02, nous sommes passés à la vitesse supérieure en ajoutant une **capacité prédictive** à notre API. Concrètement, nous avons :

- Étendu notre API FastAPI existante avec un endpoint `/train_model` :
 - Il reçoit un DataFrame au format JSON et le nom de la colonne cible ;
 - Il normalise les données avec StandardScaler et entraîne un VotingRegressor (régression linéaire, forêt aléatoire, SVR) ;
 - Il sauvegarde le pipeline entraîné avec joblib et renvoie immédiatement les **prédictions** sous forme de liste.
- Développé un front-end Vue.js (Vite) permettant de :
 - Charger un jeu de données,
 - Appeler `/train_model`,
 - Afficher les prédictions dans un graphique interactif.
- Veillé à l'**accessibilité** (norme WCAG, navigation au clavier, polices adaptées).

Ce rapport détaille nos choix d'algorithmes, l'architecture de l'API et du front-end, ainsi que les résultats obtenus et les bonnes pratiques d'accessibilité

Contexte et Objectifs

I. Contexte

Après avoir validé le MSPR 01 avec une solution ETL + API + dashboard Power BI pour explorer des données historiques de pandémies, l'OMS nous a sollicité pour passer à la phase 2 : prédire l'évolution de futurs indicateurs de santé publique.

II. Fonctions principales

- Entraînement et prédiction IA
 - Endpoint /train_model (FastAPI) acceptant un DataFrame JSON et un nom de colonne cible
 - Normalisation des données (StandardScaler)
 - Entraînement d'un VotingRegressor (régression linéaire, forêt aléatoire, SVR)
 - Sauvegarde du pipeline entraîné et retour immédiat des prédictions
- Interface utilisateur Vue.js
 - Chargement d'un jeu de données utilisateur
 - Appel à /train_model et réception des résultats
 - Affichage interactif des prédictions (graphique)
- Documentation et support
 - Génération automatique de la doc OpenAPI (Swagger/ReDoc)
 - Guide d'utilisation simple pour un public non-technique

III. Contraintes et exigences

- **Accessibilité** : conformité WCAG (contraste, navigation clavier, labels clairs)
- **Performance & scalabilité** : réponse de l'API < 500 ms, pipeline réutilisable sans ré-entraîner à chaque appel
- **Sécurité** : validation Pydantic des payloads, gestion d'erreurs explicites (HTTP 400/500)
- **Maintenabilité** : code modulaire (ETL, API, IA, front), tests unitaires/documentés

Rappel sur les données utilisées

Pour construire et entraîner notre IA, nous nous appuyons sur les jeux de données nettoyés lors du MSPR 01.

Voici l'inventaire :

I. Sources brutes (dossier SourceData)

Ces fichiers proviennent du « HIV-AIDS dataset » sur Kaggle (WHO + Unesco) et ont déjà subi un premier nettoyage (minuscules, suppression des doublons, unification des noms de pays, suppression des lignes incomplètes) :

- art_coverage_by_country_clean.csv
- Couverture des traitements antirétroviraux (ART) par pays
- art_pediatic_coverage_by_country_clean.csv
- Couverture ART pédiatrique
- no_of_cases_adults_15_to_49_by_country_clean.csv
- Nombre de cas VIH chez les adultes (15-49 ans)
- no_of_deaths_by_country_clean.csv
- Nombre de décès liés au VIH
- no_of_people_living_with_hiv_by_country_clean.csv
- Population vivant avec le VIH
- prevention_of_mother_to_child_transmission_by_country_clean.csv
- Taux de prévention de la transmission mère-enfant

II. Tables finales (dossier DatasetClean)

Après normalisation, typage et création de référentiels, les données ont été organisées en tables prêtes à charger dans la base SQL.



Fichier CSV	Table SQL	Description
pays_clean.csv	pays_clean	Liste des pays et région WHO
unite_clean.csv	unite_clean	Référentiel des unités (pourcentage, nombre, etc.)
type_statistique_clean.csv	type_statistique_clean	Types de statistiques (taux, cas, etc.)
type_traitement_clean.csv	type_traitement_clean	Types de traitements (adultes, enfants)
table_population_hiv.csv	table_population_hiv	Populations VIH (min, médian, max)
table_mortalite.csv	table_mortalite	Données de mortalité (min, médian, max)
table_statistique.csv	table_statistique	Statistiques diverses (valeur, type, unité)
table_traitement.csv	table_traitement	Couverture des traitements
table_transmission_mere_enfant.csv	table_transmission_mere_enfant	Données transmission mère-enfant

Modèle IA développé

I. Algorithmes choisis

Pour maximiser la robustesse de nos prédictions, nous avons opté pour un VotingRegressor qui combine trois modèles complémentaires :

- **LinearRegression** (régression linéaire simple)
- **RandomForestRegressor** (100 arbres, pour capturer relations non linéaires)
- **SVR** (Support Vector Regressor, bon pour gérer le bruit et les outliers)

Cette approche d'ensemble permet de limiter les biais propres à chaque algorithme et d'obtenir une meilleure généralisation.

II. Préparation des données

Avant tout apprentissage, le payload JSON reçu est transformé en DataFrame Pandas.

Puis

Suppression des colonnes non pertinentes

- region, nom_pays, sous_region, id_unite (issues des tables de référence)

Sélection automatique des variables numériques

```
X = df.drop(columns=[...], errors="ignore")
X = X.select_dtypes(include=["number"])
y = df[target_column]
```

Standardisation

Dans le pipeline, on applique StandardScaler pour centrer et réduire chaque variable, garantissant une mise à l'échelle homogène.

III. Entraînement et sauvegarde

Le modèle est construit et entraîné comme suit :

Création d'un pipeline scindé en deux étapes :

- ("scaler", StandardScaler())
- ("model", VotingRegressor(estimators=[("lr", LinearRegression()), ("rf", RandomForestRegressor(n_estimators=100)), ("svr", SVR())]))

Appel à pipeline.fit(X, y)

Sauvegarde du pipeline entraîné dans un fichier voting_regressor.pkl via joblib, pour réutilisation sans retraining systématique.

L'endpoint POST /train_model exécute cette procédure et renvoie immédiatement les prédictions sur le même jeu de données.

IV. Métriques de performance

Dans le module prediction.py, la fonction train_voting_regressor réalise :

Séparation 80/20 des données (train_test_split)

Entraînement sur X_train, y_train

Prédiction sur X_test

Calcul et affichage console :

- **MSE** (Mean Squared Error)
- **RMSE** (racine de MSE)
- **R²** (coefficient de détermination)

Visualisation Matplotlib comparant valeurs réelles vs. prédictions

Ces métriques, bien que non renvoyées par l'API, ont été utilisées en développement pour évaluer et ajuster le modèle avant intégration.

Intégration dans l'API FastAPI

I. Architecture de l'API

La couche API est organisée dans le dossier API/, avec quatre modules principaux :

- main.py
 - Point d'entrée de l'application, instantiation de FastAPI(title="MSPR API", version="1.0.0")
 - Configuration du **CORS** via CORSMiddleware pour autoriser les appels depuis le front (origines, méthodes et headers ouverts)
 - Définition des événements FastAPI (startup pour créer les tables)
 - Regroupement de tous les **endpoints CRUD** (pays, population, mortalité, traitement, statistique...)
- database.py
 - Création de l'engine asynchrone et de la session get_db() (méthode Depends)

- models.py
 - Définition des modèles SQLAlchemy (classes héritant de Base) reflétant les tables :
 - Pays, PopulationHIV, Mortalite, TransmissionMereEnfant, Traitement, Statistique
- schemas.py
 - Définition des schémas Pydantic pour valider et sérialiser les requêtes et réponses de chaque entité

Cette organisation modulaire garantit une séparation nette :

- main.py pour la logique de routage et la sécurisation des accès
- models.py et schemas.py pour la structure de données
- database.py pour la couche persistance

II. Endpoint de prédiction

Nous avons étendu main.py avec un endpoint dédié à l'IA :

```
@app.post("/train_model/")
async def train_model(payload: dict):
    """
    Endpoint pour entraîner le modèle avec les données fournies.
    """
    dataframe_dict = payload.get("dataframe")
    target_column = payload.get("target_column") # Récupérer dynamiquement la colonne cible
    if not dataframe_dict or not target_column:
        raise HTTPException(status_code=400, detail="Données ou colonne cible manquantes")
    df = pd.DataFrame.from_dict(dataframe_dict)
    try:
        # Nettoyage : suppression des colonnes non pertinentes
        columns_to_drop = ["region", "nom_pays", "sous_region", "id_unite"]
        if target_column in columns_to_drop:
            columns_to_drop.remove(target_column)
```

```

# Définir X et y
y = df[target_column]
X = df.drop(columns=columns_to_drop + [target_column], errors="ignore")
# Garder seulement les colonnes numériques
X = X.select_dtypes(include=["number"])
if X.empty:
    raise HTTPException(status_code=400, detail="Aucune colonne numérique valide pour l'entraînement.")
# Créer pipeline avec normalisation + modèle
model = VotingRegressor(estimators=[
    ("lr", LinearRegression()),
    ("rf", RandomForestRegressor(n_estimators=100)),
    ("svr", SVR())
])
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("model", model)
])
pipeline.fit(X, y)
joblib.dump(pipeline, "voting_regressor.pkl")
predictions = pipeline.predict(X)
return {
    "labels": df["annee"].tolist() if "annee" in df.columns else list(range(len(predictions))),
    "real_data": y.tolist(),
    "predicted_data": predictions.tolist()
}

```

Points clés :

- Validation **Pydantic** minimale (via levée de HTTPException)
- **StandardScaler** + **VotingRegressor** couplés dans un Pipeline
- Sauvegarde du modèle entraîné pour réutilisation
- Retour des prédictions immédiatement après entraînement

III. Documentation interactive

FastAPI génère automatiquement deux interfaces interactive grâce aux annotations de `main.py` et aux `schemas.py` :

- **Swagger UI** disponible à l'URL `/docs`
- **ReDoc** accessible via `/redoc`

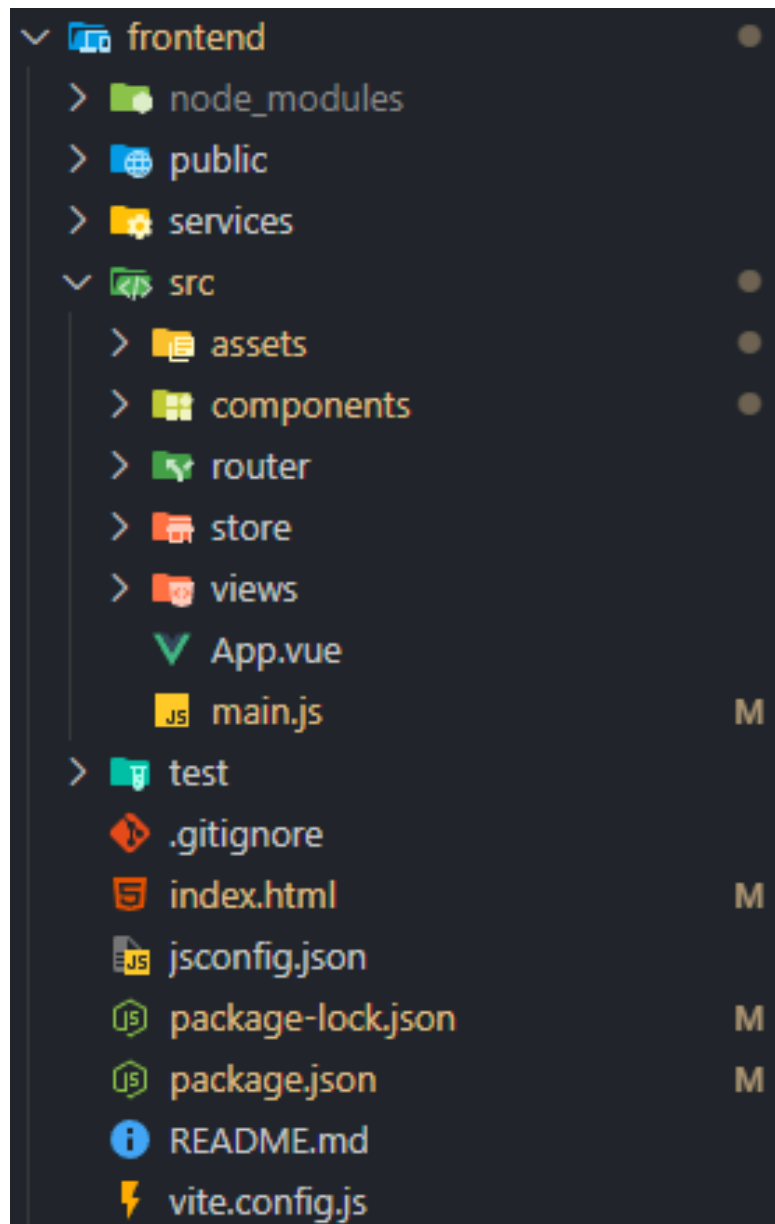
Ces interfaces permettent :

- De visualiser tous les endpoints (CRUD et IA)
- D'exécuter des requêtes de test directement depuis le navigateur
- De consulter les schémas d'entrées/sorties (automatiquement dérivés de Pydantic)

Application Front-End

I. Choix techniques

- **Framework** : Vue.js 3 (version 3.5.13)
- **Bundler** : Vite, pour des temps de build et de HMR ultrarapides
- **Langage** : JavaScript moderne (ES2021) avec configuration via `jsconfig.json`
- **Styles** : Tailwind CSS (plugin PostCSS) pour un design modulaire et un style utility-first
- **Appels API** : Axios (axios v1.8.4) pour la communication HTTP asynchrone
- **Visualisation** : Chart.js + vue-chartjs (chart.js, vue-chartjs) pour des graphiques réactifs et personnalisables
- **Routing** : Vue Router v4.5.0 pour gérer la navigation (multiples vues/pages)
- **Structure projet**



II. Fonctionnalités de l'interface

Import de données

- Formulaire pour sélectionner et charger un jeu de données (CSV ou JSON).
- Conversion en DataFrame côté front via FileReader + parsing.

Paramétrage de la prédiction

- Sélection du **target_column** (colonne cible) via un menu déroulant alimenté dynamiquement.

Appel à l'API

- Envoi du DataFrame et de la colonne cible à l'endpoint POST /train_model
- Gestion des états : chargement, succès, erreurs (400, 500)

Affichage des résultats

- Graphique linéaire interactif (Chart.js) :
 - Axe X: labels (année ou index)
 - Axe Y: valeurs prédites
- Tableau récapitulatif des prédictions

Navigation & Mise en page

- Layout responsive (desktop, tablette, mobile)
- Barre de navigation Vue Router vers d'autres fonctionnalités (tables, colonnes)

III. Accessibilité et ergonomie

La documentation « Conduite au changement dans le contexte de l'accessibilité » (dossier docs/) présente les actions menées :

- **Contraste des couleurs** respectant WCAG AA (textes sur fond, boutons, graphiques)
- **Navigation clavier** :
 - Focus visible sur tous les éléments interactifs (inputs, boutons, liens)
 - Ordre de tabulation cohérent
- **Labels explicites** pour chaque champ de formulaire (HTML <label> lié à l'<input>)
- **Lecteurs d'écran** : attributs aria-label, rôles ARIA sur les composants personnalisés
- **Police** : taille minimale de 16 px, interlignage suffisant pour faciliter la lecture

IV. Tests automatisés

- Front-end :
 - Un test Vitest est présent pour la vérification de l'affichage conditionnel selon les filtres (pays/région)
- Back-end :
 - Tests unitaires absents dans le dépôt, mais la modularité du code (services/, API/) facilite leur ajout ultérieur.

Benchmark des solutions

Pour chacun des volets du projet, nous avons comparé plusieurs options selon des critères de performance, facilité d'utilisation, compatibilité et coûts, afin de retenir la solution la plus adaptée.

I. Outils ETL

(Reprise du MSPR 01)

Outil	Performance	Simplicité d'usage	Scalabilité	Compatibilité formats	Coût
Pandas	Bonne	Très élevée	Limitée	CSV, JSON, SQL	Gratuit (Open Source)
Talend	Moyenne	Élevée	Bonne	Large (XML, etc.)	Licence payante
Apache Hop	Moyenne	Moyenne	Bonne	Connecteurs variés	Gratuit
Airflow	Élevée	Complexe	Excellente	Tous	Gratuit

Choix : Pandas pour sa simplicité, sa cohérence avec l'écosystème Python et son intégration directe dans l'API.

II. Framework Front-End

(Voir *Benchmark Vue.js _ MSPR 02.pdf*)

Framework	Performance	Documentation	Courbe d'apprentissage	Taille bundle	Écosystème Vue
Vue 3	Excellente	Très complète	Faible	↗ faible	Native (Vite)
React 18	Excellente	Excellente	Moyenne	↗ moyen	Plugins tiers
Angular	Bonne	Complète	Élevée	↗ élevé	Intégration CLI
Svelte	Excellente	Croissante	Faible	Très faible	Jeune

Choix : Vue 3 pour son intégration fluide avec Vite, sa syntaxe claire, sa documentation riche et son bundle performant.

III. Bibliothèques de visualisation

Librairie	Interactivité	Facilité d'intégration Vue	Taille	Courbe d'apprentissage
Chart.js	Bonne	Plugin <code>vue-chartjs</code>	Moyenne	Faible
D3.js	Excellente	Intégration manuelle	Élevée	Élevée
Plotly.js	Très bonne	Wrapper Vue tiers	Élevée	Moyenne

Choix : Chart.js (avec `vue-chartjs`) pour son équilibre entre fonctionnalités interactives, simplicité d'utilisation et légèreté.

IV. Choix algorithmique IA

Algorithme	Rapidité d'entraînement	Robustesse	Complexité	Besoins en data
VotingRegressor	Moyenne	Excellente	Moyenne	Moyens
Régression Linéaire simple	Très rapide	Faible	Très faible	Faibles
Forêt aléatoire	Moyenne	Bonne	Moyenne	Moyens
SVR	Lente	Moyenne	Moyenne	Moyens
Réseau de neurones (PyTorch)	Lente	Potentielle	Élevée	Élevés

Choix : un **VotingRegressor** combinant **LinearRegression**, **RandomForestRegressor** et **SVR**, pour profiter de la robustesse des forêts et de la flexibilité de **SVR**, tout en gardant un temps d'entraînement raisonnable et une intégration simple via **sklearn**.

Gestion de projet et méthodologie

Pour assurer la réussite du MSPR 02, nous avons adopté une démarche agile, découpé le projet en jalons et réparti les responsabilités de façon claire

Livrable	Responsable(s)
1. Doc détaillée : choix algorithme IA + principes ergonomie & accessibilité	Tojoniaina (structure/maquettage) ; Tojoniaina + Alessia (tests) ; Gaël (contenu IA) ; Florent (ergonomie & accessibilité)
2. Benchmark des solutions Front-end	Florent
3. Application Front-end (Vue.js + Chart.js)	Alessia
4. API IA (<code>/train_model</code> , pipeline, persistance)	Gaël
5. Documentation API OpenAPI (Swagger/ReDoc + PDF guide)	Florent ; Gaël
6. Tests automatisés & rapport de couverture (interface utilisateur)	Alessia
7. Doc conduite au changement et accessibilité	Florent

Conclusion

Ensemble, nous avons développé une solution complète et cohérente pour répondre au TPRE 502 :

- ETL et modèle de données : reprise et amélioration des scripts du MSPR 01 pour alimenter une base SQL structurée.
- API IA : extension de FastAPI avec un endpoint /train_model entraînant un VotingRegressor (LinearRegression, RandomForest, SVR), sauvegarde du pipeline et restitution immédiate des prédictions.
- Interface Front-end : application Vue 3 + Vite permettant d'importer un jeu de données, de choisir la colonne cible, d'appeler l'API et d'afficher les résultats dans un graphique interactif.
- Documentation et accessibilité :
 - Documentation OpenAPI (Swagger/ReDoc) et guide PDF pour l'API
 - Benchmark des solutions ETL, front-end, IA et visualisation
 - Guide WCAG pour garantir contraste, navigation clavier et lisibilité
- Gestion de projet : méthode agile avec backlog, sprints hebdomadaires, revues régulières et répartition claire des responsabilités.
- Tests automatisés : plan d'implémentation en cours pour renforcer la fiabilité de l'interface (Jest, Cypress).

Ce projet démontre notre capacité à concevoir et déployer une application IA de bout en bout : de la préparation des données à la prévision, en passant par l'expérience utilisateur et la documentation.

Perspectives d'évolution :

- Enrichir le pipeline IA avec des modèles plus sophistiqués (boosting, réseaux de neurones).
- Automatiser les tests back-end et front-end pour garantir une couverture complète.
- Transformer l'application en PWA ou y ajouter des notifications en temps réel.
- Intégrer des sources de données additionnelles (épidémies émergentes, facteurs socio-économiques).

Cette base solide ouvre la voie à de futures itérations, pour une plateforme toujours plus performante, accessible et orientée décisionnel.

Annexes

```
def prepare_data_generic(df, target_column):
    print("Colonnes disponibles :", df.columns)

    if "annee" in df.columns:
        df.set_index("annee", inplace=True)
    else:
        print("⚠ Aucune colonne 'annee' trouvée. L'index ne sera pas modifié.")

    # puis suite du traitement...
    X = df.select_dtypes(include=["int", "float"]).drop(columns=[target_column], errors="ignore")
    y = df[target_column]
    return X, y
```

```
def train_voting_regressor(model, X, y):
    """
    Entraîne un modèle VotingRegressor avec les caractéristiques (X) et la cible (y).
    Évalue le modèle après l'entraînement.
    """

    # Séparer les données en ensemble d'entraînement et de test
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
    print(f"Taille de X_train : {X_train.shape}, Taille de y_train : {y_train.shape}")
    print(f"Taille de X_test : {X_test.shape}, Taille de y_test : {y_test.shape}")

    # Entraîner le modèle
    print("🚀 Entraînement du modèle...")
    model.fit(X_train, y_train)
    print("✅ Modèle entraîné avec succès !")

    # Faire des prédictions
    print("📊 Évaluation du modèle...")
    y_pred = model.predict(X_test)

    # Calculer les métriques
    print(y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)

    print(f"📋 Résultats :")
    print(f"Root Mean Squared Error (RMSE) : {rmse:.2f}")
    print(f"R² score : {r2:.2f}")
```

Test Vitest

% Coverage report from v8					
File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	43.83	76.66	47.05	43.83	
services	100	100	100	100	
api.js	100	100	100	100	
src	0	50	50	0	
App.vue	0	100	100	0	2-29
main.js	0	0	0	0	1-19
src/components	57.72	83.33	45.45	57.72	
Footer.vue	0	100	100	0	3-11
Header.vue	0	0	0	0	1-16
TestPrediction.vue	62.87	86.36	44.44	62.87	1-31,185-191,199-243
src/router	0	0	0	0	
index.js	0	0	0	0	1-16
src/store	0	50	50	0	
index.js	0	0	0	0	1-6
predictionStore.js	0	100	100	0	2-16
src/views	0	100	100	0	
home.vue	0	100	100	0	3-14

Les tests automatisés ont été mis en place avec Vitest.

Le composant TestPrediction.vue, cœur de l'interface utilisateur, affiche 62.87% de couverture des lignes et 86.36% des branches conditionnelles, ce qui garantit un comportement fiable sur les interactions critiques (filtrage pays/région, formulaire, affichage graphique).

Un rapport de couverture complet a été généré pour validation.